AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model

Invariant Knowledge Substrate for Deterministic AIOS Governance

Takuya Sogawa

2026-05-23

Contents

AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model

Invariant Knowledge Substrate for Deterministic AIOS Governance

Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
Version: v0.2.0
Publication date: 2026-05-23
DOI: https://doi.org/10.5281/zenodo.20306539

Canonical language: English
Companion translation: Japanese
Series position: AIKernel / AIOS Phase-1 Specification Series, Part I-1: Knowledge Substrate
Target: AIKernel.NET Core / ROM
Proposed namespace: `AIKernel.Abstractions.Rom`
Stability: Experimental / Non-normative

Abstract

Autonomous AI systems based on Large Language Models (LLMs) depend heavily on the quality, identity, and provenance of the knowledge injected into their inference context. In many current systems, prompts, instructions, retrieved documents, and policy fragments are ultimately concatenated as unstructured strings. This prompt-as-string pattern improves flexibility, but it weakens identity, provenance, replayability, and governance.

This paper defines the **ROM Format and Knowledge Snapshot Model** as the first knowledge-substrate paper in the AIKernel / AIOS Phase-1 specification series. ROM, or **Relation-Oriented Markdown**, is treated as an immutable and governable unit of knowledge. It is not merely a Markdown convention. In AIKernel, ROM is the unit from which deterministic knowledge snapshots, semantic identity, relation topology, signature verification, and replayable context reconstruction are derived.

The central proposal is a **Formal Knowledge Identity Theory**. The paper defines canonicalization as an **Entropy Rejection Layer**: an explicit projection from representation-variant text into a canonical knowledge state. Within the limits of the canonicalization rules, two ROM documents are considered operationally identical when they project to the same canonical state and therefore produce the same semantic hash. This model does not claim to solve general natural-language semantic equivalence. Instead, it defines a deterministic identity boundary for governed AI systems.

The model introduces Causal Semantic Identity, bounded relation topology, fail-closed reference resolution, signature verification over canonical knowledge states, and deterministic context reconstruction. These mechanisms establish the knowledge substrate required by later Phase-1 papers on storage, admissibility governance, trajectory governance, execution, delegation, implementation, and unified architecture.

Keywords

AIKernel; AIOS; Relation-Oriented Markdown; ROM; Knowledge Snapshot; Semantic Hash; Causal Semantic Identity; Deterministic Governance; Replayable Context; Prompt Governance; Knowledge Graph Governance; Fail-Closed; Content Addressing; AI Operating System

---

1. Introduction

LLM-based systems increasingly operate not as isolated text generators, but as components of autonomous or semi-autonomous software systems. They retrieve documents, consume instructions, execute tools, delegate actions, and update memory. In such systems, the question is no longer only what a prompt says. The more important question is whether the system can determine **which knowledge was used**, **where it came from**, **whether it was signed**, **how it was referenced**, and **whether the same context can be reconstructed later**.

Traditional prompt engineering often collapses these concerns into a single unstructured buffer. The prompt may contain rules, examples, retrieved passages, tool descriptions, user intent, and hidden system instructions, but after concatenation the system loses the boundary between them. Once the boundary is lost, provenance, trust, topology, replayability, and access control become difficult to enforce.

AIKernel approaches this problem from an operating-system perspective. In an OS, executable code, memory pages, file handles, permissions, and process boundaries are structured resources. They are not arbi-

trary strings. Likewise, in an AI Operating System, knowledge must become a governed substrate rather than an uncontrolled text blob.

This paper defines ROM as the basic unit of that substrate. A ROM document is a structured knowledge artifact with identity, metadata, body content, relation references, and optional cryptographic signature. Multiple ROM documents can be resolved into a bounded knowledge topology and assembled into a deterministic knowledge snapshot. That snapshot can then be used by later governance layers as a stable precondition for inference.

2. Problem Statement: The Limits of Prompt-as-String

The prompt-as-string paradigm treats context construction as string concatenation. It is easy to implement, but it introduces several structural problems.

2.1 Identity Loss through Representation Variance

Whitespace, newline style, YAML key ordering, invisible characters, and non-semantic formatting changes can alter byte-level hashes. A purely byte-addressed system may therefore treat two operationally equivalent knowledge artifacts as unrelated. Conversely, an unstructured prompt may hide meaningful changes inside a large buffer where no stable logical identity is available.

AIKernel requires a governed identity model that is neither naive byte equality nor unconstrained natural-language similarity. ROM identity is therefore defined through canonicalization: a controlled projection into a canonical knowledge state.

2.2 Provenance Loss

Dynamically concatenated prompts make it difficult to answer questions such as:

- Who authored this instruction?
- When was this document signed?
- Which trust anchor was used?
- Which version of a rule participated in a prior inference?
- Which knowledge fragments were included in a replayed context?

Without provenance, an AI system cannot provide strong auditability or deterministic replay.

2.3 Referential Uncertainty and Topology Failure

Prompt fragments often refer to other documents, policies, examples, tools, or memory entries. If those references are resolved dynamically without bounded depth, cycle detection, or fail-closed behavior, the resulting knowledge graph may become unstable.

A missing reference may silently remove an important policy. A circular reference may create uncontrolled expansion. A poisoned reference may inject untrusted knowledge. These are not merely parsing errors; they are governance failures.

3. Design Goals

The ROM Format and Knowledge Snapshot Model has four design goals.

1. **Deterministic knowledge identity.** A ROM document must have a stable operational identity derived from its canonical state.
2. **Provenance preservation.** Authorship, version, signature, and trust metadata must remain attached to the knowledge unit.
3. **Bounded topology.** Relation resolution must enforce acyclicity, bounded depth, and fail-closed behavior.

4. **Replayable context reconstruction.** A prior inference context must be reconstructable from versioned knowledge snapshots.

Together, these goals shift AI context construction from prompt concatenation to governed knowledge assembly.

4. Scope and Boundary

This paper is **Paper 01** of the AIKernel / AIOS Phase-1 specification series. It defines the knowledge substrate. Later papers depend on this substrate but add separate responsibilities.

Phase / Paper	Responsibility in the AIOS architecture
Paper 01: ROM Format and Knowledge Snapshot Model	Knowledge identity and entropy reduction
Paper 02: VFS Architecture and Semantic Storage Model	Storage and transport boundary governance
Paper 03: Pre-Inference Admissibility Governance	Execution admissibility before inference
Paper 04: Trajectory Governance Model	Runtime boundedness and convergence monitoring
Papers 05-08	Execution, delegation, reference implementation, and unified architecture
Phase-2 Theory	Semantic Compilation Architecture

This paper does not define the full VFS, execution model, tool-risk model, or semantic compiler. It defines the invariant knowledge object those later layers depend on.

5. Core Model: Semantic-Addressed Knowledge

ROM is inspired by content-addressed systems such as Git and Merkle DAG structures, but it adapts the idea for governed AI knowledge. Git is content-addressable at the object level; ROM introduces an operational semantic address within explicitly defined canonicalization rules.

A ROM document is addressed not by its raw textual presentation alone, but by the canonical knowledge state produced from that presentation. This distinction matters because AI governance must preserve semantically relevant structure while rejecting non-semantic representation variance.

The term **semantic-addressed** in this paper has a precise operational meaning:

> A ROM document is semantic-addressed when its identity is derived from a canonical representation that preserves declared semantic fields, relation topology, and provenance while rejecting explicitly non-semantic variance.

This is not a general theorem of natural-language meaning. It is a deterministic engineering contract for governed knowledge artifacts.

6. Formal Definitions

6.1 ROM State

The state of a ROM document is defined as:

$$S_{rom} = \langle ID, \tau, M, B, R, \sigma \rangle$$

where:

- ID is the logical identifier of the document.
- τ is the semantic type of the document, such as Prompt, Rule, Policy, Material, or Specification.

- M is metadata, typically represented by YAML front matter.
- B is the body content.
- R is the set of relation references to other ROM documents.
- σ is an optional cryptographic signature over the canonical state.

6.2 Canonicalization as an Entropy Rejection Layer

Let f_{canon} be the canonicalization function. It maps representation-variant ROM input into a canonical knowledge state:

$$f_{canon} : S_{rom} \to S_{canon}$$

Canonicalization is defined as an **Entropy Rejection Layer**. It intentionally removes non-semantic variance while preserving the fields that contribute to governed identity.

Preserved	Rejected
Declared semantic fields	Whitespace-only variance
Relation topology	Non-semantic formatting noise
Provenance and signature-relevant metadata	Key-order instability after canonical sorting
Logical identity	Representation variance outside the canonical contract

Operational semantic equivalence is defined as:

$$S_a \equiv_{rom} S_b \iff f_{canon}(S_a) = f_{canon}(S_b)$$

The equivalence relation $\equiv_{rom}$ should be read as ROM-level canonical equivalence, not unrestricted natural-language equivalence.

6.3 Semantic Hash and Causal Semantic Identity

The semantic hash is computed over the canonical state:

$$h_{sem} = f_{hash}(f_{canon}(S_{rom}))$$

This hash establishes **Causal Semantic Identity**: the identity of a knowledge artifact remains stable across transport, repository relocation, and replay, provided that the canonicalization algorithm, resolver version, trust configuration, and referenced content versions are the same.

$$ID(v_{local}) = ID(v_{remote}) = ID(v_{replay}) = f_{hash}(f_{canon}(v))$$

This is the identity foundation required for deterministic replay.

6.4 Bounded Knowledge Topology

A ROM document may reference other ROM documents. Let $R(d)$ be the relation set of document d. Reference resolution is governed by the following constraints:

Constraint	Governance purpose
Directed acyclic topology	Prevent circular reference expansion
Bounded depth	Prevent unbounded context growth

Constraint	Governance purpose
Fail-closed missing-reference handling	Prevent silent policy loss
Versioned resolution	Preserve replayability
Trust-aware inclusion	Prevent untrusted knowledge injection

Formally, for each reference:

$$\forall r \in R(d), \quad Resolve(r) \neq \emptyset \wedge Depth(r) \leq D_{max}$$

A topology that fails these constraints is not partially accepted. It is blocked fail-closed.

6.5 Mapping to Formal Execution Semantics

ROM identity is analogous to the invariant component of formal program reasoning. The mapping to Hoare-style reasoning is as follows:

Hoare-style concept	AIKernel governance concept
Precondition	Pre-inference admissibility
Invariant	ROM semantic identity and trusted knowledge topology
Postcondition	Bounded runtime state and replayable outcome

This mapping is disciplinary rather than literal. It shows that AIKernel treats knowledge identity as a formal precondition for governed execution.

7. Governance and Security Considerations

ROM becomes a security-aware substrate because identity, provenance, and topology are checked before inference.

A signed ROM document is verified by applying canonicalization, computing the semantic hash, and validating the signature against an accepted trust anchor. Canonicalization does not make cryptographic hash collisions impossible. Rather, it prevents representation-level perturbations, such as irrelevant whitespace or key-order changes, from bypassing governance by creating meaningless identity variation.

The threat model is summarized below.

Threat	Mitigation
Prompt injection through tampered policy text	Signed canonical semantic hash verification
Reference poisoning	Trust-aware and fail-closed topology resolution
Missing policy references	Fail-closed reference validation
Replay tampering	Snapshot hashes and causal semantic identity
Hidden representation perturbation	Canonicalization before hashing and signing

8. Determinism and Replayability

Deterministic replay requires more than saving the final prompt. It requires saving the identity of every knowledge unit that participated in context construction.

Let $H_{snapshot}$ be the hash of a knowledge snapshot. Let $C_{runtime}$ be the runtime context reconstructed from that snapshot. AIKernel defines hydration as a deterministic function:

$$C_{runtime} = \Phi_{hydrate}(H_{snapshot})$$

The function is deterministic under a fixed resolver version, canonicalization version, trust configuration, and set of referenced ROM states. This transforms context construction from ephemeral string assembly into **Replayable Infrastructure**.

9. Implementation Architecture

The theoretical model maps to a minimal set of C# interface contracts under the proposed namespace `AIKernel.Abstractions.Rom`.

Interface	Responsibility
`IRomDocument`	Immutable DTO representing S_{rom}
`IRomCanonicalizer`	Produces S_{canon} and rejects representation entropy
`ISemanticHasher`	Computes h_{sem} from canonical state
`IRomSignatureVerifier`	Verifies signatures against trusted anchors
`IRomValidator`	Validates schema and invariants
`IRelationResolver`	Resolves references and enforces topology constraints
`IKnowledgeSnapshotBuilder`	Builds deterministic knowledge snapshots

A minimal pipeline is:

```
Parse ROM
  -> Canonicalize
  -> Hash
  -> Verify signature
  -> Resolve relations
  -> Validate topology
  -> Build knowledge snapshot
  -> Hydrate runtime context
```

10. Limitations

This model has several limitations.

First, ROM-level equivalence is not full semantic equivalence. It is equivalence under a declared canonicalization contract. Natural-language meaning remains broader than any deterministic canonical form.

Second, canonicalization intentionally discards non-semantic representation entropy. Prompt behaviors that depend on microscopic whitespace or formatting perturbations are not preserved. This is intentional: AIKernel treats such behaviors as unstable prompt manipulation rather than governable architecture.

Third, the security model depends on correct implementation of canonicalization, hashing, signature validation, relation resolution, and trust-store management. Bugs in those components can undermine the governance boundary.

Fourth, bounded topology may reject knowledge graphs that are useful but too large or too dynamic. This trade-off is deliberate. AIKernel prioritizes deterministic governance over unbounded context expansion.

11. Relationship to Other Phase-1 Papers

Paper 01 provides the knowledge substrate for the rest of Phase-1.

- Paper 02 governs the storage and transport layer that persists and retrieves ROM documents.
- Paper 03 uses trusted knowledge snapshots as preconditions for admissibility decisions.
- Paper 04 uses stable root goals and governed context as inputs to trajectory monitoring.
- Papers 05 and 06 rely on ROM-derived contracts during execution and delegation.

- Paper 07 validates the model through AIKernel.NET implementation.
- Paper 08 integrates the Phase-1 architecture as a unified AIOS governance model.

12. Conclusion

The ROM Format and Knowledge Snapshot Model defines the invariant knowledge substrate for AIKernel / AIOS Phase-1. It replaces prompt-as-string context assembly with governed knowledge artifacts that carry identity, provenance, relation topology, and replayable snapshot semantics.

By defining canonicalization as an entropy rejection layer, ROM establishes deterministic operational identity without claiming unrestricted natural-language semantic equivalence. By combining semantic hashes, signatures, bounded relation topology, and deterministic hydration, ROM provides the foundation on which later AIKernel governance layers can enforce admissibility, monitor inference trajectories, and reconstruct prior contexts.

In short, ROM is the knowledge substrate that allows AIKernel to treat inference context as a governed system resource rather than an accidental string.

References

1. Hoare, C. A. R. “An Axiomatic Basis for Computer Programming.” Communications of the ACM, vol. 12, no. 10, 1969, pp. 576-580. DOI: 10.1145/363235.363259.
2. Cousot, Patrick, and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, 1977.
3. Merkle, Ralph C. “A Digital Signature Based on a Conventional Encryption Function.” Advances in Cryptology - CRYPTO ’87, Lecture Notes in Computer Science, vol. 293, 1988, pp. 369-378. DOI: 10.1007/3-540-48184-2_32.
4. Git Project. “Git Internals - Git Objects.” Pro Git, Git Documentation.
5. Sikka, Varin, and Vishal Sikka. “Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models.” arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
6. IPFS Documentation. “Merkle Directed Acyclic Graphs (DAG).” IPFS Docs.
7. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.